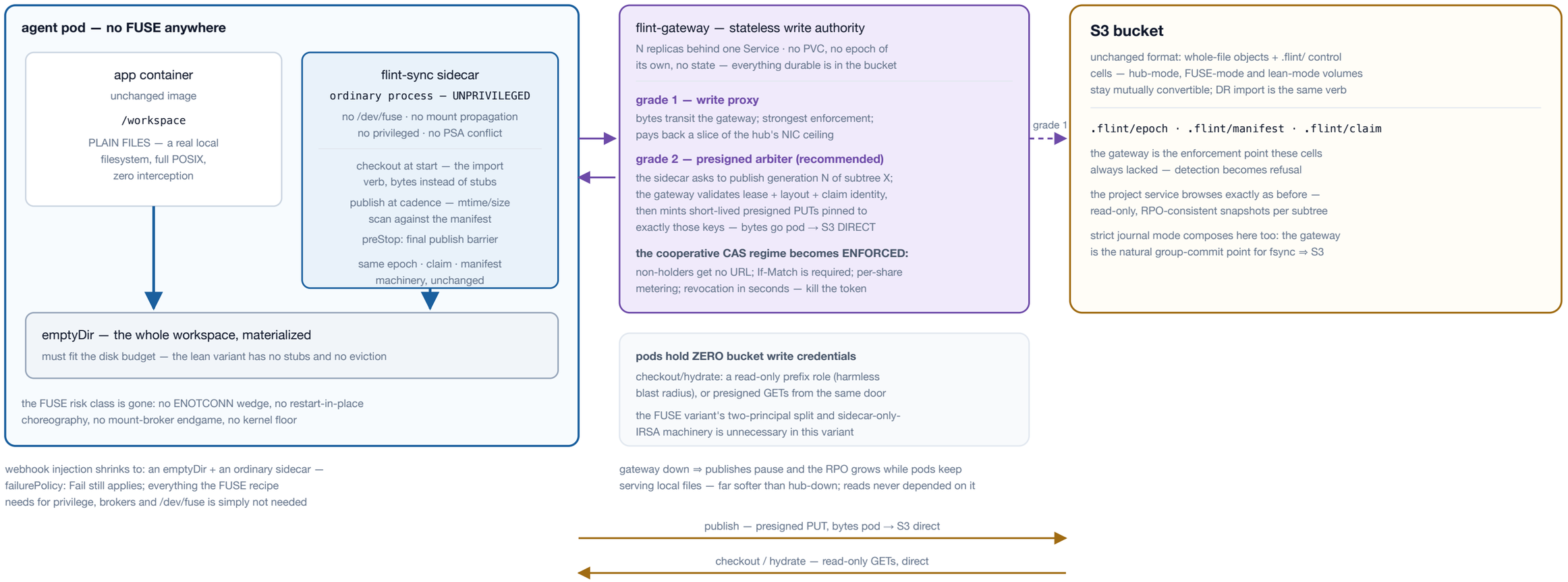
Flint-lean — plain files, one bucket

The third front end over the flint bucket format, split out of the flint-fuse architecture doc: if the workspace fits on disk, no interception is needed at all. Materialize plain files at pod start, publish changed files at cadence with an unprivileged sidecar, and put write authority in a stateless gateway. Most of FUSE's value at a fraction of its risk surface — same bucket format, same engine.



what the lean variant gives up — FUSE is the escalation, not the default

lazy hydration
cold start downloads the whole workspace; fine at GiB scale — a partial-checkout path list covers bigger trees

interception-grade capture
scan-based publish re-uploads touched files whole — the engine's whole-dirty fallback was already the correctness backstop

eviction / space management
no stubs: the workspace must fit the emptyDir budget; TiB-scale or lazy volumes are FUSE's territory (its own doc)

This is a third front end, not a FUSE configuration — the app touches plain files on a real local filesystem, and the sidecar is an ordinary unprivileged program running the same engine verbs: `import` at start (bytes instead of stubs), the flush barrier's whole-file lane at cadence (driven by an mtime/size scan against the manifest instead of capture interception), the same epoch, claim, and manifest machinery. For the agent-workspace shape — code, configs, artifacts, GiB-scale — it is most of FUSE's value with none of FUSE's hard parts: the entire privileged/mount-propagation/ENOTCONN/mount-broker risk section of the FUSE architecture stops applying, and the kernel floor drops to nothing. **The gateway restores the security posture the review said direct mode gives up.** With grade 2, "the bucket trusts one principal" is true again — pods hold no write credentials, the cooperative CAS regime becomes enforced arbitration (a non-holder is refused a URL, not merely detected after the fact), revocation returns to seconds, and per-share metering exists again. Its failure mode is publish-pause with growing RPO while pods keep serving — the softest failure shape any component on these pages has. **One dialect everywhere:** this also argues for converging the hub's file API onto the S3 REST surface (the libflint plan's §8 sidecar adapter, built once, placed three ways — hub listener, localhost sidecar, gateway), so the live view and the published view speak the same protocol and the same tools work against both. **Recommendation:** lean is the direct-mode entry point; FUSE is the escalation for volumes too large or too lazy to materialize; the hub is unchanged. Same bucket, three front ends, one engine. Implementation plan: docs/plans/flint-lean-plan.md.

One dialect — the S3 surface, three placements

The decision of record: every REST surface in the system speaks the S3 dialect — the hub's file API converges onto it, the gateway serves it, and direct-mode non-mount access (lean and FUSE alike) is plain S3 against the bucket. One library, three placements; the consistency contracts say which view a client is reading.

s3_surface — built once

`SigV4 (header + presigned) · bearer`

`ListObjectsV2 · Get/Put/Head/Delete · Range`

`If-Match · If-None-Match:* · DeleteObjects`

`multipart: parts 1–10000 · ≥5 MiB · reorder`

consolidates libflint §8's localhost-gateway floor — one implementation instead of three

conformance kit runs against AWS and MinIO

the store's bucket-wide MinIO listing workaround regains its server-side Prefix here

hub :8080 — the LIVE view

file API v2: strong, no RPO lag, atomic rename
dual-serves /files for one release, then deprecates
every data op counts as activity — polls do not

flint-gateway — the fleet door

grade-1 S3 proxy with policy, plus the ONE non-S3 verb: the grade-2 publish grant (presigned mint)
bearer auth · seconds-scale revocation

localhost S3 sidecar — later

apps that already speak S3, zero kernel path — libflint §8 adapter 3: the same library, placed a third way

extensions the dialect adds — each one deliberate

`RenameObject`	the Express One Zone verb, served on general buckets: ATOMIC on the hub's live view — something no real GP S3 offers
`x-amz-meta-flint-*`	posix stamps (mode/uid/gid/mtime) — the A12 convention
`activity classes`	data ops wake and keep a share awake; HEAD and list polls do not — a polling browser must not pin a share Ready forever
`publish grant`	POST intent → presigned set — grade 2's only custom verb

the multipart traps are requirements, not surprises

the ETag is md5-of-part-md5s + "-N", never a content MD5 — the s3proxy #338 class
CompleteMultipartUpload can return 200 WITH an error in the body — unhandled, SDK retries corrupt state
complete-by-concatenation costs 3× IO — reserve-and-copy is an explicit decision, since part sizes are unknown until receipt

two views, one protocol — the consistency contracts tell them apart

the hub endpoint — the LIVE view

strong: close-to-open, locks behind it, atomic rename, no RPO lag — reads and writes land on the working set through the coherence authority

who: front doors, editors, anything touching a LIVE shared tree

the bucket endpoint — the PUBLISHED view

RPO-consistent snapshots per subtree; untorn whole-file generations; read-only against live subtrees (a console PUT has three bad fates — consistency doc)

who: project browsers, DR, analytics, direct-mode non-mount access

direct-mode REST access — lean AND FUSE variants alike — is the bucket endpoint: plain S3, no flint server anywhere in the read path.
the same rclone / boto3 / aws-cli invocation works against either endpoint; only the URL and the consistency contract change.

Why S3 and not a nicer bespoke API: the ecosystem is the feature — every language has a mature S3 client, presigned flows are understood by every CI system and browser uploader, and the tools people already point at buckets (rclone, boto3, cyberduck, Velero) work against a live flint share the day the dialect ships. The file API was already shaped on S3 semantics — whole-file GET/PUT with ETag and If-Match (the v1.30 conditional-writes work maps one-to-one onto S3's own conditional headers) — so this is a dialect change, not an architecture change. **The one thing the hub's live view offers that no real S3 can:** assembly under a temp name completed by an atomic RENAME under a deny-both OPEN — an atomic, exclusive multipart-complete, the differentiator the libflint plan already claims for the localhost gateway, inherited here. **Migration is dual-serve:** the hub answers both `/files` and the S3 dialect for one release; the front door moves; `/files` deprecates. Bearer auth stays alongside SigV4 so existing gateway tokens keep working, and revocation keeps its seconds-scale semantics. The activity classification is the one place the dialect must be flint-aware from day one — the idle ladder's lesson stands: a conditional GET answering 304 pins a share exactly as hard as a 200, so what counts as "activity" is declared per verb, not inferred.